



ADMINISTACIÓN DE BASE DE DATOS

PRÁCTICA DE LABORATORIO

Tema: Transacciones distribuidas, protocolo 2PC, transparencia y replicación.

Contexto del escenario:

Sistema de Base de Datos Distribuida, donde la información contable de una entidad financiera está fragmentada horizontalmente por criterios geográficos:

- Nodo Central (Quito): Aloja la base de datos BD_Quito con las cuentas de los clientes de la región Sierra.
- Nodo Remoto (Guayaquil): Aloja la base de datos BD_Guayaquil con las cuentas de los clientes de la región Costa.

El caso de negocio simula una Transferencia Interbancaria de 1,000 USD, que requiere una operación síncrona y distribuida que consiste en debitar en el saldo de una cuenta en la Sierra (Quito) y acreditar simultáneamente en una cuenta de la Costa(Guayaquil).

Pasos para la práctica:

1. Crear dos bases de datos en la misma instancia para simular dos ciudades (BD_Quito y BD_Guayaquil).

```
CREATE DATABASE BD_Quito;  
GO  
USE BD_Quito;
```

```
CREATE TABLE Cuentas_Quito (  
    id_cuenta INT PRIMARY KEY,  
    cliente VARCHAR(50),  
    saldo DECIMAL(12,2) NOT NULL CHECK (saldo >= 0)  
);
```

```
INSERT INTO Cuentas_Quito VALUES (1001, 'Ana Martínez (Sierra)', 1500.00);  
GO
```

```
CREATE DATABASE BD_Guayaquil;  
GO  
USE BD_Guayaquil;
```

```
CREATE TABLE Cuentas_Guayaquil (  
    id_cuenta INT PRIMARY KEY,  
    cliente VARCHAR(50),  
    saldo DECIMAL(12,2) NOT NULL CHECK (saldo >= 0)  
);
```



```
INSERT INTO Cuentas_Guayaquil VALUES (2002, 'Carlos Ruiz (Costa)', 500.00);  
GO
```

2. Lograr la Transparencia de Fragmentación, regresamos a la base de datos principal (BD_Quito) y creamos una Vista Unificada. El usuario final solo consultará la vista sin saber que los datos están divididos físicamente.

```
USE BD_Quito;  
GO
```

```
CREATE VIEW V_CUENTAS_GLOBAL AS  
SELECT id_cuenta, cliente, saldo, 'Quito' AS Ubicacion_Fisica FROM  
BD_Quito.dbo.Cuentas_Quito  
UNION ALL  
SELECT id_cuenta, cliente, saldo, 'Guayaquil' AS Ubicacion_Fisica FROM  
BD_Guayaquil.dbo.Cuentas_Guayaquil;  
GO
```

```
-- El usuario consulta todo el banco sin saber que está dividido en dos  
ciudades
```

```
SELECT * FROM V_CUENTAS_GLOBAL;
```

id_cuenta	cliente	saldo	Ubicacion_Fisica
1001	Ana Martinez (Sierra)	1500.00	Quito
2002	Carlos Ruiz (Costa)	500.00	Guayaquil

3. Utilizamos bloques BEGIN DISTRIBUTED TRANSACTION y control de errores con TRY...CATCH, para preparar el escenario de transferencias bancarias.

Escenario A. Transacción Exitosa

```
USE BD_Quito;  
GO
```

```
BEGIN TRY  
    -- Iniciamos la transacción distribuida (Invoca internamente a 2PC)  
    BEGIN DISTRIBUTED TRANSACTION;  
  
    -- 1. Nodo Quito: Debitar 1000 USD  
    UPDATE BD_Quito.dbo.Cuentas_Quito  
    SET saldo = saldo - 1000  
    WHERE id_cuenta = 1001;  
  
    -- 2. Nodo Guayaquil: Acreditar 1000 USD  
    UPDATE BD_Guayaquil.dbo.Cuentas_Guayaquil  
    SET saldo = saldo + 1000  
    WHERE id_cuenta = 2002;  
  
    -- Si ambas operaciones fueron exitosas, el coordinador ejecuta la Fase  
2: COMMIT  
    COMMIT TRANSACTION;  
    PRINT 'Transacción Distribuida Exitosa con 2PC.';  
END TRY  
BEGIN CATCH  
    -- Si algún nodo falla (no acepta o error de red), se ejecuta ROLLBACK  
    global
```



```
ROLLBACK TRANSACTION;  
PRINT 'Error detectado. Se ejecutó ROLLBACK global para garantizar  
consistencia.';  
END CATCH;
```

```
-- Verificación de saldos  
SELECT * FROM BD_Quito.dbo.Cuentas_Quito;  
SELECT * FROM BD_Guayaquil.dbo.Cuentas_Guayaquil;
```

id_cuenta	cliente	saldo
1001	Ana Martínez (Sierra)	500.00

id_cuenta	cliente	saldo
2002	Carlos Ruiz (Costa)	1500.00

Escenario B: Simulación de Falla

```
USE BD_Quito;  
GO  
  
BEGIN TRY  
    BEGIN DISTRIBUTED TRANSACTION;  
  
    -- Intentamos debitar 2000 USD  
    UPDATE BD_Quito.dbo.Cuentas_Quito  
    SET saldo = saldo - 2000  
    WHERE id_cuenta = 1001;  
  
    -- Esta instrucción en Guayaquil teóricamente es válida, pero no debe  
    consolidarse  
    UPDATE BD_Guayaquil.dbo.Cuentas_Guayaquil  
    SET saldo = saldo + 2000  
    WHERE id_cuenta = 2002;  
  
    COMMIT TRANSACTION;  
END TRY  
BEGIN CATCH  
    -- El CATCH atraparé la violación del CHECK constraint de Quito  
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;  
    PRINT 'Transacción abortada con éxito. Razón del fallo: ' +  
    ERROR_MESSAGE();  
END CATCH;  
  
-- Comprobación, los saldos deben seguir intactos (atomicidad)  
SELECT * FROM BD_Quito.dbo.Cuentas_Quito;  
SELECT * FROM BD_Guayaquil.dbo.Cuentas_Guayaquil;
```



id_cuenta	cliente	saldo
1001	Ana Martínez (Sierra)	500.00

id_cuenta	cliente	saldo
2002	Carlos Ruiz (Costa)	1500.00

4. Comprobemos que el protocolo 2PC congela recursos mientras espera que todos los nodos voten.

Ventana A (El Coordinador se congela)

```
USE BD_Quito;
GO
BEGIN DISTRIBUTED TRANSACTION;

UPDATE BD_Quito.dbo.Cuentas_Quito
SET saldo = saldo - 100
WHERE id_cuenta = 1001;
-- NOTA: Dejar la pestaña abierta sin ejecutar COMMIT ni ROLLBACK.
```

Ventana B (El recurso bloqueado)

```
USE BD_Quito;
GO
SELECT * FROM Cuentas_Quito WHERE id_cuenta = 1001;
```

(Para liberar el entorno, regresa a la Ventana A y ejecutar ROLLBACK TRANSACTION;).

5. El paso final de la arquitectura es encapsular toda la complejidad distributiva y el manejo del TRY...CATCH dentro de un objeto de base de datos. Así, el programador del sistema o cajero del banco logra la transparencia total de fallos y transacciones. Crear el objeto en BD_Quito.

```
USE BD_Quito;
GO

CREATE PROCEDURE sp_ProcesarTransferenciaInterbancaria
    @id_origen INT,
    @id_destino INT,
    @monto DECIMAL(12,2)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN DISTRIBUTED TRANSACTION;

        -- 1. Cargo a la cuenta de la Sierra
        UPDATE BD_Quito.dbo.Cuentas_Quito
        SET saldo = saldo - @monto
        WHERE id_cuenta = @id_origen;
```



```
-- 2. Abono a la cuenta de la Costa
UPDATE BD_Guayaquil.dbo.Cuentas_Guayaquil
SET saldo = saldo + @monto
WHERE id_cuenta = @id_destino;

COMMIT TRANSACTION;
PRINT '¡Transferencia procesada con éxito global!';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
    RAISERROR ('Error en la transferencia distribuida. Red de servidores
segura. Detalle: %s', 16, 1, @ErrMsg);
END CATCH
END;
GO
```

6. Prueba de Transparencia, una transferencia compleja se reduce a una sola línea de código abstracta.

```
EXEC sp_ProcesarTransferenciaInterbancaria 1001, 2002, 100.00;
```

id_cuenta	cliente	saldo
1001	Ana Martínez (Sierra)	400.00

id_cuenta	cliente	saldo
2002	Carlos Ruiz (Costa)	1600.00

7. Simulación de replicación total

El banco tiene una tabla llamada SUCURSALES con los códigos, direcciones y teléfonos de todas las oficinas del país. Como esta información casi nunca cambia y todos los cajeros la necesitan constantemente, se decide clonar la tabla completa tanto en BD_Quito como en BD_Guayaquil.

```
-- Creamos la tabla original en el Nodo Central (Quito)
USE BD_Quito;
GO
```

```
CREATE TABLE Sucursales_Quito_Origen (
    id_sucursal INT PRIMARY KEY,
    nombre_sucursal VARCHAR(50),
    ciudad VARCHAR(30)
);
```

```
-- Insertamos los datos globales en el origen
INSERT INTO Sucursales_Quito_Origen VALUES
(1, 'Agencia Terminal Terrestre', 'Riobamba'),
(2, 'Agencia Mall del Río', 'Cuenca'),
(3, 'Agencia Centro Histórico', 'Quito');
GO
```

```
--Creamos la La Réplica Total en Guayaquil
USE BD_Guayaquil;
```



GO

```
CREATE TABLE Sucursales_Guayaquil_Replica (  
    id_sucursal INT PRIMARY KEY,  
    nombre_sucursal VARCHAR(50),  
    ciudad VARCHAR(30)  
);  
GO
```

--TRÁFICO DE REPLICACIÓN

```
INSERT INTO BD_Guayaquil.dbo.Sucursales_Guayaquil_Replica  
SELECT * FROM BD_Quito.dbo.Sucursales_Quito_Origen;  
GO
```

-- Se verán todas las sucursales del país localmente

```
SELECT * FROM BD_Guayaquil.dbo.Sucursales_Guayaquil_Replica;
```

id_sucursal	nombre_sucursal	ciudad
1	Agencia Terminal Terrestre	Riobamba
2	Agencia Mall del Río	Cuenca
3	Agencia Centro Histórico	Quito

8. Simulación de Replicación Parcial

En el banco cada ciudad maneja sus propios clientes locales. Sin embargo, el banco tiene una regla: Los clientes VIP deben poder ser atendidos en cualquier ciudad del país de forma inmediata y sin latencia. Por lo tanto, el banco decide que solo las cuentas con saldos mayores o iguales a 1,000 USD se replicarán en las otras provincias; las cuentas normales se quedan en su base de datos local.

--Preparamos el terreno en Guayaquil para recibir la réplica parcial de Quito

```
USE BD_Guayaquil;
```

GO

```
CREATE TABLE Clients_VIP_Quito_En_Gye (  
    id_cuenta INT PRIMARY KEY,  
    cliente VARCHAR(50),  
    saldo DECIMAL(12,2)  
);  
GO
```

--SIMULACIÓN DEL FILTRO DE REPLICACIÓN PARCIAL (Criterio: Saldo >= 1000)

-- El motor analiza BD_Quito, pero SOLO copia las cuentas VIP hacia Guayaquil

```
INSERT INTO BD_Guayaquil.dbo.Clients_VIP_Quito_En_Gye (id_cuenta, cliente, saldo)  
SELECT id_cuenta, cliente, saldo  
FROM BD_Quito.dbo.Cuentas_Quito  
WHERE saldo >= 1000.00;  
GO
```

-- 3. Verificación del Filtro Parcial

```
SELECT * FROM BD_Guayaquil.dbo.Clients_VIP_Quito_En_Gye;
```

--Insertar un nuevo registro en la sierra para que se pase con la replicación

```
USE BD_Quito;
```

GO

```
INSERT INTO Cuentas_Quito VALUES (1002, 'Luis Narváez (Sierra)', 1200.00);
```

GO

```
SELECT * FROM Cuentas_Quito;
```

--Limpiamos la tabla espejo que está en Guayaquil

```
TRUNCATE TABLE BD_Guayaquil.dbo.Clients_VIP_Quito_En_Gye;
```

GO

--Verificamos nuevamente



```
SELECT * FROM BD_Guayaquil.dbo.Clients_VIP_Quito_En_Gye;
```

id_cuenta	cliente	saldo
1002	Luis Narváez (Sierra)	1200.00

Actividades para realizar luego de la práctica:

Actividad 1.

El banco ahora exige que la replicación parcial sea bidireccional. Es decir, Guayaquil también debe enviar sus clientes VIP (saldo>1,000 USD) hacia Quito para que la Sierra tenga sus datos localmente.

Tareas a realizar:

1. Crea una tabla en BD_Quito llamada Clients_VIP_Gye_En_Quito.
2. Realiza un INSERT en BD_Guayaquil.dbo.Cuentas_Guayaquil de un nuevo cliente de la Costa (María Belén, id: 2003) con un saldo inicial de 1,400.00 USD.
3. Escribe el script de replicación parcial que extraiga los datos VIP desde Guayaquil y los inserte en la nueva tabla de Quito.
4. Ejecuta un SELECT en Quito para demostrar que el nuevo cliente VIP de la Costa ya es visible en la Sierra.

Actividad 2.

En el laboratorio se aprendió que el comando INSERT INTO ... SELECT es un "Snapshot manual. Si un saldo cambia, la réplica se queda desactualizada. El reto es obligar al motor a mantener la consistencia en tiempo real.

Automatizar la Replicación Total de Sucursales. Cada vez que el Administrador inserte una nueva sucursal en el origen (BD_Quito), esta debe replicarse en el mismo milisegundo en el nodo remoto (BD_Guayaquil) de forma automática sin que el usuario digite código extra.

Tareas a realizar:

1. Crea un disparador (AFTER INSERT TRIGGER) en la tabla Sucursales_Quito_Origen.



2. El trigger debe incluir internamente un bloque distributivo que inserte el nuevo registro en BD_Guayaquil.dbo.Sucursales_Guayaquil_Replica.
3. Inserta la "Agencia Norte" en Quito y demuestra mediante un SELECT en Guayaquil que apareció mágicamente.

Enviar el script a la siguiente carpeta en la nube, nombrar al archivo Apellido_Nombre:

https://unachedu-my.sharepoint.com/:f:/g/personal/raquel_moyano_unach_edu_ec/lgBhQG4sFHCNQIsaSTPf9A5NAZQ2x3Vue73umWFQBq0Wd_U?e=gKFXxd